HW 7

Minh Truong

141225

Perform the following activities in Python:

- Get the dataset - pick any one of the datasets from the Dataset folder.
- Load the .csv file into a data frame in Python.
- Describe the data, provide information on the dataset.
- Preprocess data if needed.
- Conduct SVM Classification.
- Experiment with different kernel functions in SVC to see which one performs best for your data.
- Generate classification report.
- Provide metrics for precision, recall, and ROC.

SVM Classification

Dataset: Wisconsin Diagnostic Breast Cancer (WDBC) **Source:** UCI Machine Learning Repository **Target:** Diagnosis (B = Benign, M = Malignant)

In [36]:
```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import confusion_matrix, accuracy_score, classificatior
```

Get the Dataset - pick any one of the datasets from the Dataset folder.

I chose the Wisconsin Diagnostic Breast Cancer (WDBC) dataset. It has 569 instances, 30 features, no missing values.

Load the .csv file into a data frame in Python.

In [37]:
```
#Define column names because wdbc.data has no header. The names were sourced
columns = ['id', 'diagnosis',
           'radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean',
           'smoothness_mean', 'compactness_mean', 'concavity_mean', 'concave
           'symmetry_mean', 'fractal_dimension_mean',
           'radius_se', 'texture_se', 'perimeter_se', 'area_se',
           'smoothness_se', 'compactness_se', 'concavity_se', 'concave_point
           'symmetry_se', 'fractal_dimension_se',
           'radius_worst', 'texture_worst', 'perimeter_worst', 'area_worst',
           'smoothness_worst', 'compactness_worst', 'concavity_worst', 'conc
           'symmetry_worst', 'fractal_dimension_worst']

# Load dataset into dataframe df
df = pd.read_csv('/Users/mtruong/python-projects/BZAN 6355/wdbc.data', heade

# Provides first 5 rows of dataset
df.head()
```

Out[37]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	sr
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

5 rows × 32 columns

Describe the data, provide information on the dataset.

In [38]:
```
# Provides the summary statistics: count, mean, std, min, max for all featur
df.describe()
```

Out[38]:

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smo
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	

8 rows × 31 columns

In [51]:

```
# Check that there no missing values
df.isnull().sum()
```

Out[51]:

```
diagnosis                  0
radius_mean                0
texture_mean               0
perimeter_mean             0
area_mean                  0
smoothness_mean            0
compactness_mean           0
concavity_mean             0
concave_points_mean        0
symmetry_mean              0
fractal_dimension_mean     0
radius_se                  0
texture_se                 0
perimeter_se               0
area_se                    0
smoothness_se              0
compactness_se             0
concavity_se               0
concave_points_se          0
symmetry_se                0
fractal_dimension_se       0
radius_worst               0
texture_worst              0
perimeter_worst            0
area_worst                 0
smoothness_worst           0
compactness_worst          0
concavity_worst            0
concave_points_worst       0
symmetry_worst             0
fractal_dimension_worst    0
dtype: int64
```

In [52]:
```
# Class distribution: 357 Benign, 212 Malignant
df['diagnosis'].value_counts()
```

Out[52]:
```
diagnosis
0    357
1    212
Name: count, dtype: int64
```

Preprocess the data if needed.

In [41]:
```
# Drop ID, encode target, scale features

# Drop the ID column as it is not a predictive feature
df.drop(columns=['id'], inplace=True)

# Encode the target variable: B = 0 (Benign), M = 1 (Malignant)
le = LabelEncoder()
df['diagnosis'] = le.fit_transform(df['diagnosis'])

# Define the feature matrix and target vector
X = df.drop(columns=['diagnosis'])
y = df['diagnosis']

# Split the dataset into 80% train, 20% test, and stratify to preserve class
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, ra

# Standardize features to fit on train only to prevent data leakage into tes
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Experiment with different kernel functions in SVC to see which one performs best for your data.

In [49]:
```
# Test four kernels: linear, rbf, poly, and sigmoid kernels to find the best
kernels = ['linear', 'rbf', 'poly', 'sigmoid']
for kernel in kernels:
    model = SVC(kernel=kernel, random_state=42)
    model.fit(X_train, y_train)
    print(f"Kernel: {kernel:<10} Accuracy: {accuracy_score(y_test, model.pre
```

```
Kernel: linear     Accuracy: 0.9649
Kernel: rbf        Accuracy: 0.9737
Kernel: poly       Accuracy: 0.8860
Kernel: sigmoid    Accuracy: 0.9474
```

In [43]:
```
# Train the final model using the best kernel from the comparison above
svm = SVC(kernel='rbf', probability=True, random_state=42)
svm.fit(X_train, y_train)
y_pred = svm.predict(X_test)
```

Generate classification report.

In [50]:
```
# Provide the full breakdown of precision, recall, and F1-score per class
print(classification_report(y_test, y_pred, target_names=['Benign', 'Maligna
```

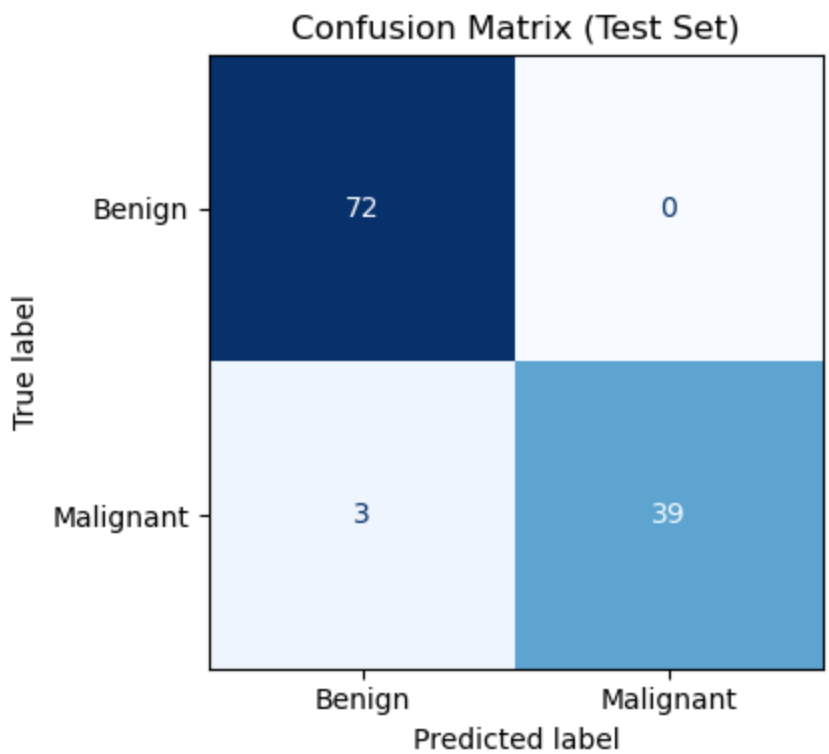
```
              precision    recall  f1-score   support

      Benign       0.96      1.00      0.98        72
   Malignant       1.00      0.93      0.96        42

    accuracy                           0.97       114
   macro avg       0.98      0.96      0.97       114
weighted avg       0.97      0.97      0.97       114
```

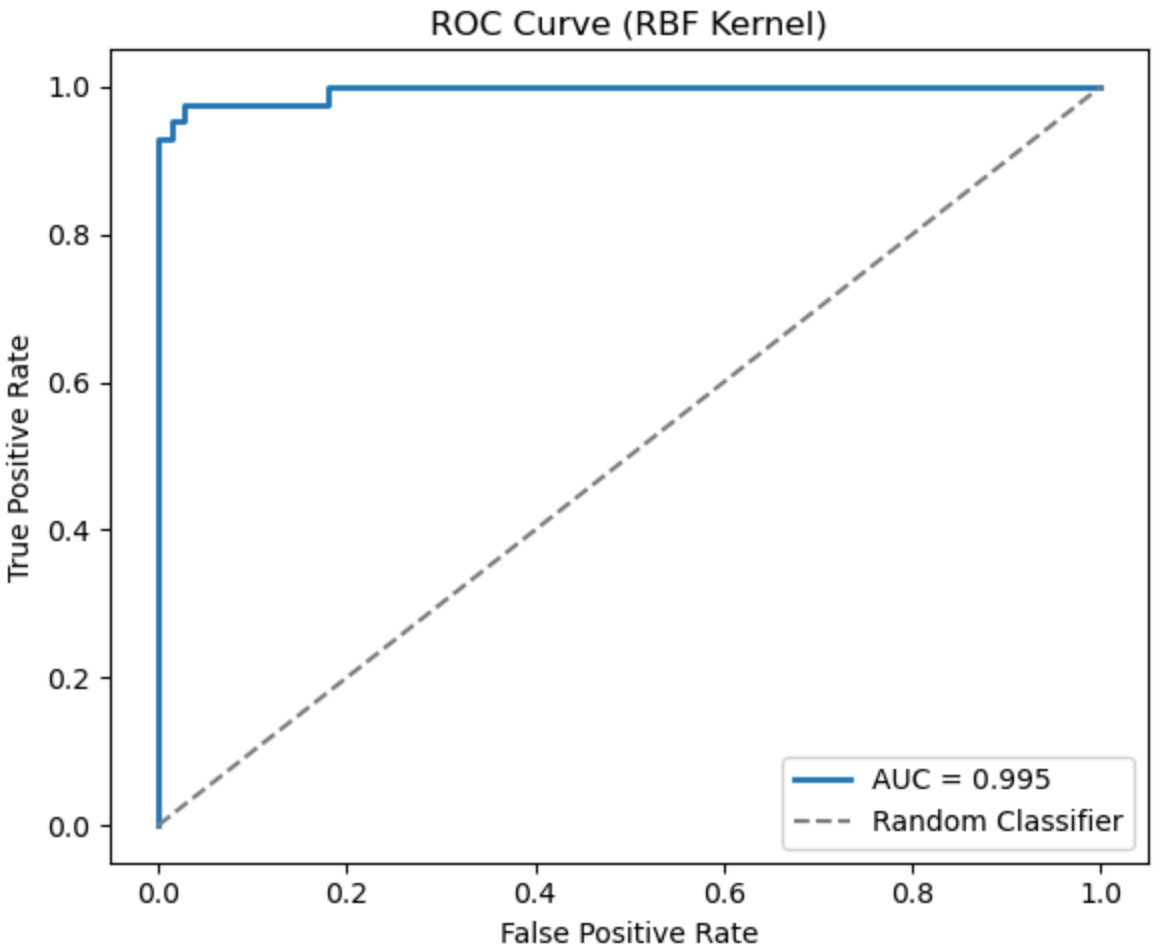
Provide metrics for precision, recall, and ROC.

In [46]:
```
# Provide the confusion matrix on the test set to compare predicted vs actua
fig, ax = plt.subplots(figsize=(5, 4))
ConfusionMatrixDisplay(confusion_matrix(y_test, y_pred),
                       display_labels=['Benign', 'Malignant']).plot(ax=ax, c
ax.set_title('Confusion Matrix (Test Set)')
plt.tight_layout()
plt.show()
```



In [47]:
```
y_proba = svm.predict_proba(X_test)[:, 1]
fpr, tpr, _ = roc_curve(y_test, y_proba)
roc_auc = auc(fpr, tpr)
```

```
plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, lw=2, label=f'AUC = {roc_auc:.3f}')
plt.plot([0, 1], [0, 1], color='grey', linestyle='--', label='Random Classif
plt.title('ROC Curve (RBF Kernel)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.tight_layout()
plt.show()
```



Conclusion: The RBF kernel outperformed all other kernels on this breast cancer dataset. It achieved the highest accuracy and an AUC-ROC score of 0.995. This confirms the effectiveness of SVMs for high-dimensional medical classification tasks.